

An Analysis of Encoder decoder architecture

Encoder decoder architecture

Encoder decoder architecture -- Part 1

In words, it means that each token can pay attention to itself, and every token before it, but not any after it. A non-masked attention module can be thought of as a masked attention module where the mask has all entries zero. As an example of an uncommon use of mask matrix, the XLNet considers all masks of the form $P \cdot M$ causal $P - 1$ $\{\displaystyle PM_{\{\text{causal}\}}P^{-1}\}$, where P $\{\displaystyle P\}$ is a random permutation matrix.

Encoder decoder architecture -- Part 2

where FFN $\{\displaystyle \{\text{FFN}\}\}$ stands for "feed-forward network". We can more succinctly write it as $\text{EncoderLayer}(H) = \text{FFN}(\text{MultiheadAttention}(H, H, H))$ $\{\displaystyle \{\text{EncoderLayer}\}(H) = \{\text{FFN}\}(\{\text{MultiheadAttention}\}(H, H, H))\}$ with the implicit convention that the FFN $\{\displaystyle \{\text{FFN}\}\}$ is applied to each row of the matrix individually. The encoder layers are stacked. The first encoder layer takes the sequence of input vectors from the embedding layer, producing a sequence of vectors. This sequence of vectors is processed by the second encoder, and so on. The output from the final encoder layer is then used by the decoder. As the encoder processes the entire input all at once, every token can attend to every other token (all-to-all attention), so there is no need for causal masking.

Encoder decoder architecture -- Part 3

Sub-quadratic transformers Training transformer-based architectures can be expensive, especially for long inputs. Many methods have been developed to attempt to address the issue. In the image domain, Swin transformer is an efficient architecture that performs attention inside shifting windows. In the audio domain, SepTr decouples the attention in time and frequency domains. Long Range Arena (2020) is a standard benchmark for comparing the behavior of transformer architectures over long inputs.

Encoder decoder architecture -- Part 4

KV caching When an autoregressive transformer is used for inference, such as generating text, the query vector is different at each step, but the already-computed key and value vectors are always the same. The KV caching method saves the computed key and value vectors at each attention block, so that they are not recomputed at each new token. PagedAttention applies memory paging to KV caching. If a transformer is used with a baked-in prompt, such as ["You are a customer support agent..."], then the key and value vectors can be computed for the prompt, and saved on disk. The saving in compute is significant when the model is

used for many short real-time interactions, such as in online chatbots. In general, when a user uses an autoregressive transformer to generate a continuation to a sequence of tokens, the model would first perform a forward-pass on this sequence, whereby the KV caches over this sequence are computed. This is called prefilling. Hyperscalers serving large Transformer models may use disaggregated inference, wherein prefilling and decoding are performed on separately specialized hardware.

Encoder decoder architecture -- Part 5

This has a neutral effect on model quality and training speed, but increases inference speed. More generally, grouped-query attention (GQA) partitions attention heads into groups, each of which shares the key-value pair. MQA is GQA with one group, while standard Multihead Attention is GQA with the maximal number of groups.

Encoder decoder architecture -- Part 6

FlashAttention FlashAttention is an algorithm that implements the transformer attention mechanism efficiently on a GPU. It is a communication-avoiding algorithm that performs matrix multiplications in blocks, such that each block fits within the cache of a GPU, and by careful management of the blocks it minimizes data copying between GPU caches (as data movement is slow). See the page on softmax for details. An improved version, FlashAttention-2, was developed to cater to the rising demand for language models capable of handling longer context lengths. It offers enhancements in work partitioning and parallelism, enabling it to achieve up to 230 TFLOPs/s on A100 GPUs (FP16/BF16), a 2x speed increase over the original FlashAttention. Key advancements in FlashAttention-2 include the reduction of non-matmul FLOPs, improved parallelism over the sequence length dimension, better work partitioning between GPU warps, and added support for head dimensions up to 256 and multi-query attention (MQA) and grouped-query attention (GQA). Benchmarks revealed FlashAttention-2 to be up to 2x faster than FlashAttention and up to 9x faster than a standard attention implementation in PyTorch. Future developments include optimization for new hardware like H100 GPUs and new data types like FP8. FlashAttention-4 focuses on pipelining to increase instruction throughput, and was developed to perform particularly well on Blackwell GPUs.

Encoder decoder architecture -- Part 7

In deep learning, the transformer is a family of artificial neural network architectures based on the multi-head attention mechanism, in which text is converted to numerical representations called tokens, and each token is converted into a

An Analysis of Encoder decoder architecture

Encoder decoder architecture

vector via lookup from a word embedding table. At each layer, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished. Because self-attention alone is permutation-invariant, transformers inject positional information, typically through positional encodings or learned positional embeddings, so token order can affect the output. Transformers have the advantage of having no recurrent units, therefore requiring less training time than earlier recurrent neural architectures (RNNs) such as long short-term memory (LSTM). Later variations have been widely adopted for training large language models (LLMs) on large (language) datasets. Modern transformer designs are commonly grouped into encoder-only, decoder-only, and encoder-decoder variants, depending on whether they are optimized for representation learning, autoregressive generation, or conditional sequence-to-sequence tasks.

Encoder decoder architecture -- Part 8

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the next token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.

Encoder decoder architecture -- Part 9

restoring or repairing incomplete or corrupted text. For example, the input, "Thank you ~~ me to your party ~~ week", might generate the output, "Thank you for inviting me to your party last week". translation between natural languages (machine translation) judging the pragmatic acceptability of natural language. For example, the following sentence might be judged "not acceptable", because even though it is syntactically well-formed, it is improbable in ordinary human usage: The course is jumping well. Note that while each of these tasks is trivial or obvious for human native speakers of the language (or languages), they have typically proved challenging for previous generations of machine learning architecture.